

# Podstawy Logiki Cyfrowej

ASK 2026

Instytut Informatyki  
Uniwersytet Wrocławski

25 lutego 2026

# Układy logiczne

---

Będziemy tworzyć układy wykonujące obliczenia na bitach.

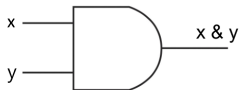
Zmienne  $x$ ,  $y$ ,  $z$ , ... będą zmiennymi binarnymi reprezentującymi ładunki (lub ich brak) przepływające przez przewody.

Podstawowym elementem z którego będziemy budować układy będą bramki logiczne.

# Bramki logiczne AND, OR, NOT

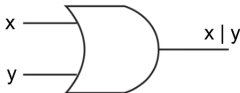
Bramka AND

| x | y | $x \& y$ |
|---|---|----------|
| 0 | 0 | 0        |
| 0 | 1 | 0        |
| 1 | 0 | 0        |
| 1 | 1 | 1        |



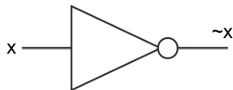
Bramka OR

| x | y | $x   y$ |
|---|---|---------|
| 0 | 0 | 0       |
| 0 | 1 | 1       |
| 1 | 0 | 1       |
| 1 | 1 | 1       |



Bramka NOT

| x | $\sim x$ |
|---|----------|
| 0 | 1        |
| 1 | 0        |



# Układy logiczne

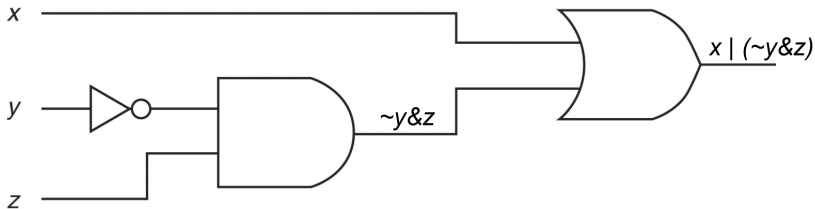
---

$$x \mid (\sim y \ \& \ z)$$

# Układy logiczne

---

$$x \mid (\sim y \& z)$$



# Inne bramki logiczne

---

Bramka XOR

| x | y | $x \oplus y$ |
|---|---|--------------|
| 0 | 0 | 0            |
| 0 | 1 | 1            |
| 1 | 0 | 1            |
| 1 | 1 | 0            |

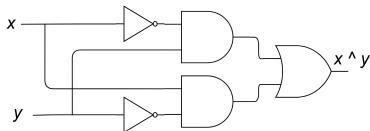


# Inne bramki logiczne

---

Bramka XOR

| x | y | $x \oplus y$ |
|---|---|--------------|
| 0 | 0 | 0            |
| 0 | 1 | 1            |
| 1 | 0 | 1            |
| 1 | 1 | 0            |

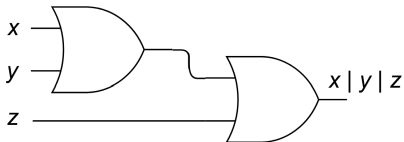
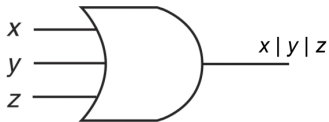


# Inne bramki logiczne

---

Trzywejściowa bramka OR

$x \mid y \mid z$





# Inne operatory logiczne

---

$$x \Rightarrow y$$

| x | y | $x \Rightarrow y$ |
|---|---|-------------------|
| 0 | 0 | 1                 |
| 0 | 1 | 1                 |
| 1 | 0 | 0                 |
| 1 | 1 | 1                 |

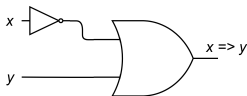
# Inne operatory logiczne

---

$$x \Rightarrow y$$

| x | y | $x \Rightarrow y$ |
|---|---|-------------------|
| 0 | 0 | 1                 |
| 0 | 1 | 1                 |
| 1 | 0 | 0                 |
| 1 | 1 | 1                 |

$$\sim x \mid y$$



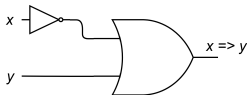
# Inne operatory logiczne

---

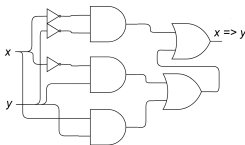
$$x \Rightarrow y$$

| x | y | $x \Rightarrow y$ |
|---|---|-------------------|
| 0 | 0 | 1                 |
| 0 | 1 | 1                 |
| 1 | 0 | 0                 |
| 1 | 1 | 1                 |

$$\sim x \mid y$$



$$\begin{aligned} &(\sim x \ \& \ \sim y) \mid \\ &(\sim x \ \& \ y) \mid \\ &(x \ \& \ y) \end{aligned}$$



# Pół sumator

---

Naszym celem jest dodanie do siebie dwóch bitów.

Bit  $s$  oznacza sumę,  $c$  to bit przeniesienia.

| $x$ | $y$ | $c$ | $s$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 1   | 0   | 1   |
| 1   | 0   | 0   | 1   |
| 1   | 1   | 1   | 0   |

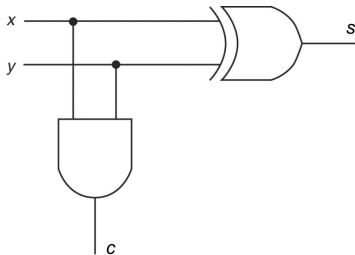
# Pół sumator

---

Naszym celem jest dodanie do siebie dwóch bitów.

Bit  $s$  oznacza sumę,  $c$  to bit przeniesienia.

| $x$ | $y$ | $c$ | $s$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 1   | 0   | 1   |
| 1   | 0   | 0   | 1   |
| 1   | 1   | 1   | 0   |



# Sumator

---

Teraz przyjmijmy, że dostajemy na wejściu również bit przeniesienia.

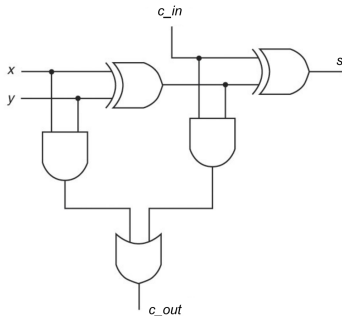
| c_in | x | y | c_out | s |
|------|---|---|-------|---|
| 0    | 0 | 0 | 0     | 0 |
| 0    | 0 | 1 | 0     | 1 |
| 0    | 1 | 0 | 0     | 1 |
| 0    | 1 | 1 | 1     | 0 |
| 1    | 0 | 0 | 0     | 1 |
| 1    | 0 | 1 | 1     | 0 |
| 1    | 1 | 0 | 1     | 0 |
| 1    | 1 | 1 | 1     | 1 |

# Sumator

---

Teraz przyjmijmy, że dostajemy na wejściu również bit przeniesienia.

| c_in | x | y | c_out | s |
|------|---|---|-------|---|
| 0    | 0 | 0 | 0     | 0 |
| 0    | 0 | 1 | 0     | 1 |
| 0    | 1 | 0 | 0     | 1 |
| 0    | 1 | 1 | 1     | 0 |
| 1    | 0 | 0 | 0     | 1 |
| 1    | 0 | 1 | 1     | 0 |
| 1    | 1 | 0 | 1     | 0 |
| 1    | 1 | 1 | 1     | 1 |



# Wielobitowy sumator

---

Aby dodać do siebie dwie liczby o zapisach bitowych  $X_{15}X_{14} \dots X_0$  oraz  $Y_{15}Y_{14} \dots Y_0$  możemy połączyć ze sobą kaskadowo 16 sumatorów.



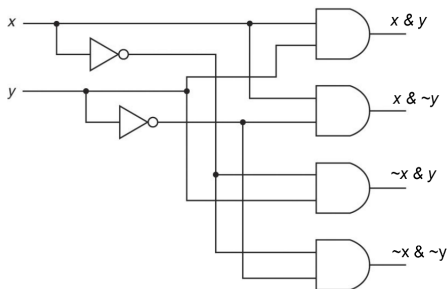


# Dekoder

---

Dekoder jest układem, który na wejściu dostaje  $n$  bitów, reprezentujących pewną liczbę  $k$ .

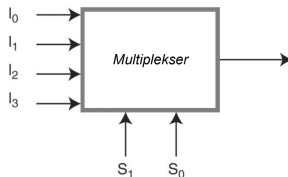
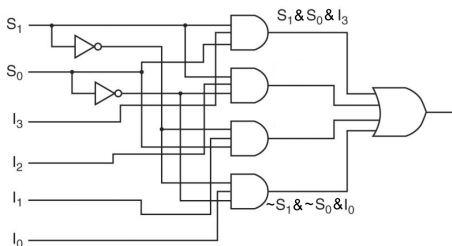
Na wyjściu znajduje się  $2^n$  bitów, z czego zapalony jest tylko ten o numerze  $k$ .



# Multiplexer

Multiplexer dostaje na wejściu  $2^n$  bitów z danymi oraz  $n$  bitów reprezentujących pewną liczbę  $S$ .

Na wyjściu znajduje się jeden bit, którego wartość jest równa  $S$ -temu spośród bitów wejściowych.



# Prosta, dwubitowa jednostka ALU

---

Stwórzmy dwubitową jednostkę arytmetyczno-logiczną.

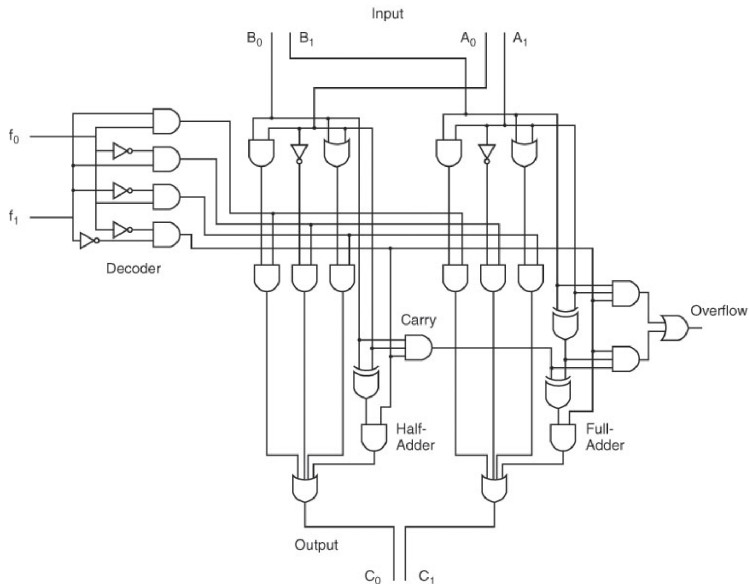
Na wejściu podane będą:

- 2 bity  $A_1$  oraz  $A_0$  reprezentujące liczbę  $A$ ,
- 2 bity  $B_1$  oraz  $B_0$  reprezentujące liczbę  $B$ ,
- 2 bity sterujące  $f_0$  oraz  $f_1$  reprezentujące liczbę  $f$ , oznaczającą numer wykonywanej operacji.

Na wyjściu otrzymamy dwa bity  $C_1$  i  $C_0$  oznaczające wynik operacji:

- $A + B$ , jeśli  $f = 0$ ,
- $\sim A$ , jeśli  $f = 1$ ,
- $A \mid B$ , jeśli  $f = 2$ ,
- $A \& B$ , jeśli  $f = 3$ .

# Prosta, dwubitowa jednostka ALU



# Przerzutnik S-R

---

Chcielibyśmy mieć układ, który przechowuje zapisaną wartość 0 lub 1.

Układ będzie posiadał swój aktualny stan Q. Będzie miał również dwa wejścia:

- S – ustaw wartość przerzutnika na 1,
- R – ustaw wartość przerzutnika na 0.

| S | R | Q               |
|---|---|-----------------|
| 0 | 0 | stan bez zmian  |
| 0 | 1 | 0               |
| 1 | 0 | 1               |
| 1 | 1 | niezdefiniowane |

# Przerzutnik S-R

---

| S | R | Q- | Q+ |
|---|---|----|----|
| 0 | 0 | 0  | 0  |
| 0 | 0 | 1  | 1  |
| 0 | 1 | 0  | 0  |
| 0 | 1 | 1  | 0  |
| 1 | 0 | 0  | 1  |
| 1 | 0 | 1  | 1  |
| 1 | 1 | 0  | -  |
| 1 | 1 | 1  | -  |

Q- oznacza poprzedni stan  
przerzutnika,

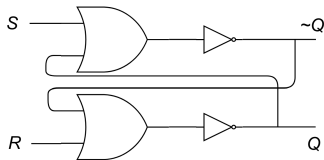
Q+ oznacza nowy stan  
przerzutnika.

# Przerzutnik S-R

---

| S | R | Q- | Q+ |
|---|---|----|----|
| 0 | 0 | 0  | 0  |
| 0 | 0 | 1  | 1  |
| 0 | 1 | 0  | 0  |
| 0 | 1 | 1  | 0  |
| 1 | 0 | 0  | 1  |
| 1 | 0 | 1  | 1  |
| 1 | 1 | 0  | -  |
| 1 | 1 | 1  | -  |

Q- oznacza poprzedni stan przerzutnika,  
Q+ oznacza nowy stan przerzutnika.



# Przerzutnik S-R sterowany poziomem sygnału

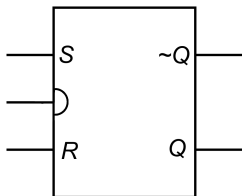
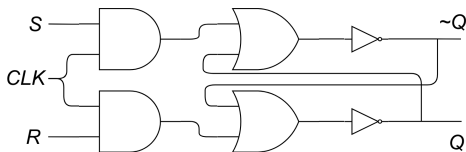
---

Stwórzmy przerzutnik, który zmienia stan tylko jeśli aktualny sygnał zegara *CLK* jest równy 1.



# Przerzutnik S-R sterowany poziomem sygnału

Stwórzmy przerzutnik, który zmienia stan tylko jeśli aktualny sygnał zegara  $CLK$  jest równy 1.



## Przerzutnik S-R sterowany zboczem sygnału

---

Możemy również stworzyć przerzutnik, który będzie zmieniał swój stan tylko w momencie, gdy zegar zmieni swój stan z 1 na 0.

